

SignAI - Complete Setup Guide for New Laptop

This guide will walk you through setting up the entire SignAI project from scratch on a new computer.

Table of Contents

1. [Prerequisites](#)
 2. [Repository Setup](#)
 3. [Backend Setup](#)
 4. [Frontend Setup](#)
 5. [Running the Project](#)
 6. [Troubleshooting](#)
 7. [Quick Start Summary](#)
-

Prerequisites

System Requirements

- **OS:** Windows 10/11, macOS, or Linux
- **RAM:** Minimum 8GB (16GB recommended for GPU training)
- **Disk Space:** 20GB minimum (for models, datasets, node_modules)
- **GPU:** Optional but recommended for inference speedup (NVIDIA CUDA)

Required Software

Before starting, install these on your new laptop:

1. **Git** - Version control
 - Download: <https://git-scm.com/download/win> (Windows)
 - Or: `brew install git` (macOS)
 - Verify: `git --version`
2. **Python** - Programming language (version 3.11 or higher)
 - Download: <https://www.python.org/downloads/>
 - **IMPORTANT:** Check "Add Python to PATH" during installation
 - Verify: `python --version`
3. **Node.js** - JavaScript runtime (version 18 or higher)
 - Download: <https://nodejs.org/> (LTS version recommended)
 - Verify: `node --version` and `npm --version`
4. **Git LFS** (Large File Storage) - For large model files
 - Download: <https://git-lfs.github.com/>
 - Install: Follow the installer

- Verify: `git lfs version`

Repository Setup

Step 1: Clone the Repository

```
# Navigate to desired location
cd Desktop

# Clone the SignAI repository
git clone https://github.com/yourusername/signai.git
cd signai
```

If using Git LFS (for large model files):

```
git lfs install
git lfs pull
```

Step 2: Verify Project Structure

After cloning, verify the folder structure:

```
signai/
├── backend/           # FastAPI + ML backend
├── frontend/          # React + Vite frontend
├── datasets/          # Training data
├── .venv/             # Virtual environment (will be created)
├── README.md          # Project overview
├── MODEL_PIPELINES.md # Detailed model architecture
├── SETUP.md           # This file
└── requirements.txt    # Python dependencies
```

Backend Setup

Step 3: Create Python Virtual Environment

A virtual environment isolates project dependencies from system Python.

```
# Navigate to project root
cd signai

# Create virtual environment
python -m venv .venv
```

```
# Activate virtual environment
# ⚠️ Different commands for Windows vs macOS/Linux

# Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# Windows Command Prompt (cmd):
.venv\Scripts\activate.bat

# macOS/Linux:
source .venv/bin/activate
```

You should see `(.venv)` prefix in your terminal after activation.

Step 4: Install Python Dependencies

```
# Upgrade pip to latest version
python -m pip install --upgrade pip

# Install all required packages
pip install -r backend/requirements.txt
```

This installs:

- **FastAPI** - REST API framework
- **PyTorch & TorchVision** - Deep learning
- **Transformers** - Hugging Face library (VideoMAE model)
- **OpenCV** - Image processing
- **Pillow** - Image manipulation
- **matplotlib & seaborn** - Visualization
- **scikit-learn** - ML metrics & evaluation
- And more...

Installation takes 10-15 minutes depending on internet speed.

Step 5: Verify Backend Installation

```
# Test Python environment
python -c "import torch; print(f'PyTorch version: {torch.__version__}')"
python -c "import transformers; print(f'Transformers version: {transformers.__version__}')"

# You should see version numbers for both
```

Step 6: Check Model Checkpoints

```
# Verify model files exist
ls backend/checkpoints/

# You should see:
# - best_model.pt
# - videomae_classes.json
```

If model files are missing, download them from the project repository or contact the team.

Frontend Setup

Step 7: Install Node.js Dependencies

```
# Navigate to frontend directory
cd frontend

# Install npm packages (takes 5-10 minutes)
npm install

# Verify installation
npm list --depth=0
```

This installs all React, TypeScript, and build tool dependencies.

Step 8: Verify Frontend Setup

```
# Check key packages installed
npm ls react typescript vite tailwindcss

# You should see version numbers for all
```

Running the Project

Option A: Run Backend Only

```
# From project root
cd backend

# Activate virtual environment (if not already active)
.\.venv\Scripts\Activate.ps1 # Windows PowerShell
# or
source ../.venv/bin/activate # macOS/Linux

# Use the convenient startup script
```

```
.\run-backend.ps1 # Windows PowerShell
# or
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# ☒ Backend runs on: http://localhost:8000
# ☐ API docs available at: http://localhost:8000/docs
```

Option B: Run Frontend Dev Server

```
# From project root
cd frontend

# Start Vite development server
npm run dev

# ☒ Frontend runs on: http://localhost:5173
```

Option C: Run Both (Recommended)

Open **two separate terminals**:

Terminal 1 - Backend:

```
cd signai/backend
.\.venv\Scripts\Activate.ps1
.\run-backend.ps1
```

Terminal 2 - Frontend:

```
cd signai/frontend
npm run dev
```

Now:

- Frontend: <http://localhost:5173>
- Backend API: <http://localhost:8000>
- API Docs: <http://localhost:8000/docs>

Option D: Build for Production

```
# Frontend build
cd frontend
npm run build
```

```
# Output goes to: frontend/out/

# Backend can be deployed with:
# - Docker
# - Cloud services (AWS, GCP, Azure)
# - Heroku
# - DigitalOcean
```

Running the Model

Laptop with GPU (NVIDIA)

If you have an NVIDIA GPU:

```
# Verify CUDA is available
python -c "import torch; print(torch.cuda.is_available())"

# Should output: True
# Then model inference runs on GPU (30-45ms latency)
```

Laptop with CPU Only

Model will run on CPU:

```
# Inference takes 200-250ms on modern CPUs
# Still functional, just slower than GPU
```

Troubleshooting

Issue 1: Python not found

```
Error: 'python' is not recognized
```

Solution:

- Reinstall Python and CHECK "Add Python to PATH"
- Restart terminal after installation
- Use `python --version` to verify

Issue 2: Virtual environment activation fails

```
Error: .venv\Scripts\Activate.ps1 is not digitally signed
```

Solution (Windows PowerShell):

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Then retry activation.

Issue 3: Port 8000 already in use

```
Error: Address already in use
```

Solution:

```
# Find process using port 8000
netstat -ano | findstr :8000 # Windows
# or
lsof -ti:8000                # macOS/Linux

# Kill the process or use different port
uvicorn app.main:app --port 8001
```

Issue 4: Node modules installation fails

```
Error: npm ERR! connection timed out
```

Solution:

```
# Clear npm cache
npm cache clean --force

# Try again
npm install
```

Issue 5: Model file not found

```
Error: FileNotFoundError: best_model.pt
```

Solution:

- Download model from repository releases
- Place in: `backend/checkpoints/best_model.pt`
- Or download via Git LFS: `git lfs pull`

Issue 6: CUDA/GPU not detected

Error: CUDA device not found

Solution (if you have NVIDIA GPU):

```
# Install CUDA 12.1 from https://developer.nvidia.com/cuda-downloads
# Install cuDNN
# Reinstall PyTorch with CUDA support
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121
```

Project Structure After Setup

After successful setup, your project should look like:

```
signai/
├── .venv/                                # Virtual environment ✓
├── backend/
│   ├── app/
│   │   ├── main.py                      # FastAPI app entry
│   │   ├── schemas.py                  # Request/response models
│   │   └── services/                   # Business logic
│   ├── training/                       # Model training code
│   ├── scripts/                        # Utility scripts
│   ├── checkpoints/
│   │   ├── best_model.pt               # ✓ Model weights
│   │   └── videomae_classes.json       # ✓ Class labels
│   ├── .env                           # Environment config ✓
│   ├── requirements.txt                # Python dependencies ✓
│   ├── run-backend.ps1                 # Quick start script ✓
│   └── run-backend.bat                 # Quick start script ✓
├── frontend/
│   ├── src/
│   │   ├── pages/                     # React pages
│   │   ├── components/                # Reusable components
│   │   ├── services/                  # API client
│   │   └── App.tsx                     # Root component
│   ├── node_modules/                  # NPM packages ✓
│   ├── package.json                   # Dependencies ✓
│   ├── vite.config.ts                 # Build config
│   └── tsconfig.json                  # TypeScript config
```



```
|  
|— datasets/                # Training data  
|— README.md               # Project overview ✓  
|— MODEL_PIPELINE.md       # Model architecture ✓  
|— SETUP.md                # This guide ✓  
|— requirements.txt         # Python dependencies (reference)
```

☑ Quick Start Summary

After everything is installed, to run SignAI:

Shortest Path (2 steps):

1. Terminal 1 - Start Backend:

```
cd signai\backend  
.\.venv\Scripts\Activate.ps1  
.\run-backend.ps1
```

2. Terminal 2 - Start Frontend:

```
cd signai\frontend  
npm run dev
```

3. Open in browser:

- <http://localhost:5173>

🔗 Next Steps

After setup is complete:

1. **Test Live Recognition:** Go to <http://localhost:5173/live> and test webcam
2. **Upload a Video:** Try <http://localhost:5173/upload> with a video file
3. **View History:** Check <http://localhost:5173/history> for predictions
4. **Explore API Docs:** Visit <http://localhost:8000/docs> for full API reference
5. **Read Documentation:** Check [MODEL_PIPELINE.md](#) for technical details

📞 Support

If you encounter issues:

1. Check the **Troubleshooting** section above
2. Review error messages carefully

3. Ensure all prerequisites are correctly installed
 4. Check `backend/.env` and `frontend/.env` if they exist
 5. Make sure ports 8000 and 5173 are not blocked by firewall
-

Backup & Version Control

After setup, initialize git if not already done:

```
# Already cloned? Skip this
git init

# Track your changes
git add .
git commit -m "Initial setup complete"

# Push to remote (if you have a repo)
git remote add origin <your-repo-url>
git push -u origin main
```

Happy coding! 🚀 SignAI is ready to recognize Indian Sign Language!